

# 01 — Index Fundamentals

*"An index is a data structure that trades storage and write overhead for faster reads."*

---

## Table of Contents

1. [Learning Objectives](#)
  2. [Why Indexes Exist](#)
  3. [The Full Table Scan Problem](#)
  4. [How an Index Works — Conceptual Model](#)
  5. [Cost Tradeoffs](#)
  6. [How the Query Planner Chooses](#)
  7. [Index Types Overview](#)
  8. [Creating and Dropping Indexes](#)
  9. [Monitoring Index Usage](#)
  10. [Common Mistakes](#)
  11. [Best Practices](#)
  12. [Performance Considerations](#)
  13. [Interview Questions & Answers](#)
  14. [Exercises with Solutions](#)
  15. [Production Scenarios](#)
  16. [Cross-References](#)
- 

## Learning Objectives

After completing this section you will be able to:

- Explain why sequential scans degrade at scale and why indexes solve that problem
  - Describe the storage and write cost tradeoffs introduced by every index
  - Explain how PostgreSQL's cost-based optimizer decides whether to use an index
  - List all eight index types available in PostgreSQL and their primary use cases
  - Create, monitor, and drop indexes using standard PostgreSQL DDL
  - Identify indexes that are unused, duplicate, or bloated in a production database
- 

## Why Indexes Exist

A relational database stores data in **heap files**: 8 KB pages laid end-to-end on disk.

When you run:

```
SELECT * FROM orders WHERE customer_id = 42;
```

Without an index PostgreSQL must read **every single page** of the `orders` table — a **sequential scan (Seq Scan)**. For a table with 100 million rows spread across 400,000 pages that means  $400,000 \times 8 \text{ KB} = \sim 3 \text{ GB}$  of I/O for one query.

An index is a **separate, smaller data structure** that maps column values to heap locations (ctid = page number + tuple offset). Instead of reading every page the planner navigates the index, finds the matching ctids, and fetches only those pages.

### Scale comparison

| Rows        | Pages (8 KB) | Seq Scan I/O | Index Scan I/O (10 matches)    |
|-------------|--------------|--------------|--------------------------------|
| 10,000      | 44           | 352 KB       | ~1 index page + 10 heap pages  |
| 1,000,000   | 4,425        | 35 MB        | ~3 index pages + 10 heap pages |
| 100,000,000 | 442,478      | 3.4 GB       | ~4 index pages + 10 heap pages |

The index lookup cost grows as  **$O(\log n)$**  while sequential scan grows as  **$O(n)$** .

### The Full Table Scan Problem

```
Table: orders (10,000,000 rows)

Page 1 [ row1 | row2 | row3 | ... | row400 ]
Page 2 [ row401 | row402 | ... ]
...
Page 25000 [ ... | row10000000 ]

Query: WHERE customer_id = 42
--> must inspect ALL 25,000 pages
--> most I/O wasted on non-matching rows
```

PostgreSQL reads pages into the **shared\_buffers** cache (default 128 MB). If the table is larger than the cache the OS must evict other pages, causing cache churn.

### How an Index Works — Conceptual Model

```
CREATE INDEX idx_orders_customer ON orders(customer_id);

Index structure (B-tree, simplified):

      [ 42 | 67 | 91 ]    <-- root/internal page
      /      |      \
[11|28|42] [67|72|85] [91|95|99] <-- leaf pages
  |         |         |
ctid list  ctid list  ctid list
(0,1)(0,2) (3,7)(3,8) (9,1)(9,2)
  |         |         |
Heap pages  Heap pages  Heap pages
```

Steps for `WHERE customer_id = 42` :

- 1. Read index root page (1 I/O)
- 2. Follow pointer to leaf page (1 I/O)
- 3. Collect ctids for customer\_id = 42
- 4. Fetch only those heap pages (n I/O, where n = matching rows / rows per page)

Total: typically 2-5 I/Os vs thousands for a Seq Scan.

---

## Cost Tradeoffs

Every index carries **three categories of cost**:

### 1. Storage Cost

- The index itself occupies disk space (and `shared_buffers` when cached).
- A B-tree index on a single integer column typically adds 30–70% of the column's raw size.
- A GIN index on a `tsvector` column can be larger than the table itself.

### 2. Write Cost (INSERT / UPDATE / DELETE)

Every write that changes an indexed column must also update the index structure.

```
INSERT INTO orders (...) -- must update N indexes where N = number of indexes on the table
UPDATE orders SET customer_id = 99 WHERE id = 1; -- updates index for customer_id
DELETE FROM orders WHERE id = 1; -- marks index entry as dead (vacuum cleans later)
```

**Rule of thumb:** Each additional index on a table reduces write throughput by ~5–15% depending on index size and page caching.

### 3. Planner Overhead

More indexes mean more query-plan possibilities for the optimizer to evaluate. This is usually negligible but can matter for queries with dozens of joins.

#### Tradeoff Summary Table

| Scenario                   | Benefit           | Cost                            |
|----------------------------|-------------------|---------------------------------|
| OLAP (mostly reads)        | Large speedup     | Acceptable write overhead       |
| OLTP (balanced)            | Moderate speedup  | Moderate write overhead         |
| Bulk loading               | None during load  | Significant if rebuilding after |
| Write-heavy (logs, events) | Small or negative | High write amplification        |

---

## How the Query Planner Chooses

PostgreSQL uses a **cost-based optimizer (CBO)**. It assigns numeric costs to each possible plan and picks the lowest-cost plan.

### Cost Units

- `seq_page_cost` = 1.0 (default, baseline)
- `random_page_cost` = 4.0 (default for HDD; set to 1.1 for SSD)
- `cpu_tuple_cost` = 0.01
- `cpu_index_tuple_cost` = 0.005
- `cpu_operator_cost` = 0.0025

### Selectivity

The planner estimates the **fraction of rows returned** by a predicate using statistics stored in `pg_statistic`.

```
selectivity = estimated_matching_rows / total_rows
```

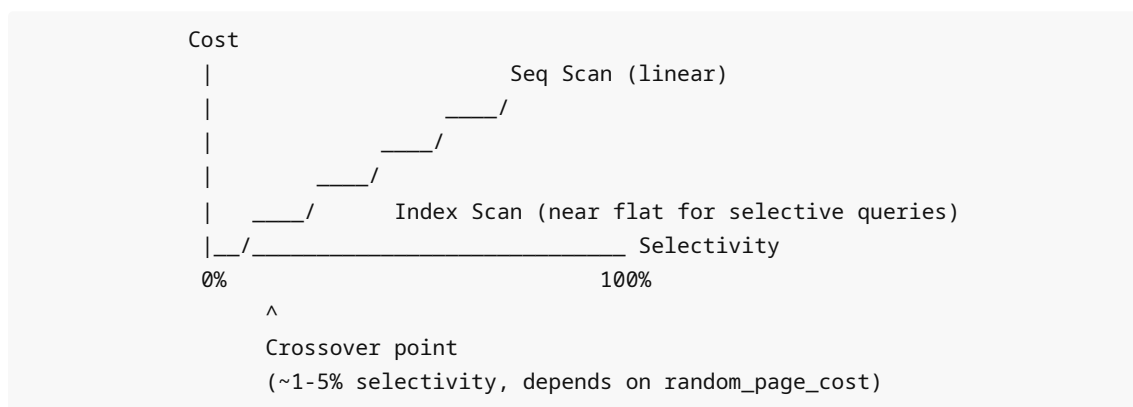
For `customer_id = 42` with 10 M rows and 200,000 distinct customers:

```
selectivity = 1 / 200,000 = 0.000005 --> very selective --> index preferred
```

For `status IN ('pending', 'processing')` where 60% of rows match:

```
selectivity = 0.60 --> not selective --> sequential scan cheaper
```

## The Crossover Point



When selectivity is **below** the crossover the index wins. When selectivity is **above** the crossover the sequential scan wins.

## Forcing the Planner (for debugging only)

```
-- Disable sequential scans to force index usage
SET enable_seqscan = off;
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
SET enable_seqscan = on;

-- Disable index scans to force sequential scan
SET enable_indexscan = off;
```

**Warning:** Never set these in production. They exist for testing and troubleshooting.

## Index Types Overview

| Type   | Best For                 | Access Methods                           |
|--------|--------------------------|------------------------------------------|
| B-tree | Equality, range, sorting | =, <, >, <=, >=, BETWEEN, LIKE 'prefix%' |
| Hash   | Equality only            | =                                        |

|         |                                    |                        |
|---------|------------------------------------|------------------------|
| GIN     | Multi-valued types                 | @>, <@, @@, &&, ?      |
| GiST    | Geometric, range, full-text        | &&, @>, <@, ~=, <<, >> |
| BRIN    | Large tables with natural ordering | =, <, >, BETWEEN       |
| SP-GiST | Hierarchical/partitioned types     | =, <, >, &&, @>        |
| Bloom   | Multi-column equality (extension)  | =                      |
| RUM     | Full-text with ranking (extension) | @@, with ranking       |

Default index type (when no type specified): **B-tree**.

## Creating and Dropping Indexes

### Basic Syntax

```
-- Default B-tree index
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- Explicit type
CREATE INDEX idx_orders_status USING hash ON orders(status);

-- Unique index
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Concurrent index (does not lock table)
CREATE INDEX CONCURRENTLY idx_orders_created ON orders(created_at);

-- Partial index (only index rows matching predicate)
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Composite index
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at DESC);

-- Expression index
CREATE INDEX idx_users_lower_email ON users(lower(email));

-- Covering index (INCLUDE)
CREATE INDEX idx_orders_cust_inc_total
ON orders(customer_id) INCLUDE (total_amount, status);
```

### Dropping Indexes

```
DROP INDEX idx_orders_customer;
DROP INDEX CONCURRENTLY idx_orders_customer; -- non-blocking
DROP INDEX IF EXISTS idx_orders_customer;
```

## Monitoring Index Usage

### Find unused indexes

```
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,           -- number of times index was used
    idx_tup_read,       -- index tuples read
    idx_tup_fetch,      -- heap tuples fetched via index
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
    AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_relation_size(indexrelid) DESC;
```

### Find duplicate indexes

```
SELECT
    a.indexname AS index1,
    b.indexname AS index2,
    a.tablename,
    a.indexdef
FROM pg_indexes a
JOIN pg_indexes b ON a.tablename = b.tablename
    AND a.indexname < b.indexname
    AND a.indexdef = b.indexdef;
```

### Index size and bloat

```
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    pg_size_pretty(pg_relation_size(indrelid)) AS table_size,
    round(100.0 * pg_relation_size(indexrelid)
        / pg_relation_size(indrelid), 2) AS pct_of_table
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
ORDER BY pg_relation_size(indexrelid) DESC;
```

### Check if index is being used in a query

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;
```

---

## Common Mistakes

### 1. Over-indexing

Adding an index on every column "just in case".

**Problem:** Slows down writes, consumes storage, confuses the planner.

**Fix:** Only index columns used in WHERE, JOIN, ORDER BY, or GROUP BY predicates that run frequently with high selectivity.

### 2. Indexing low-cardinality columns alone

```
-- BAD: status has 3 possible values, 33% selectivity each
CREATE INDEX idx_orders_status ON orders(status);
-- Planner will almost always choose Seq Scan anyway
```

**Fix:** Use partial indexes or composite indexes with higher-cardinality leading columns.

### 3. Not using CONCURRENTLY on production tables

```
-- BAD: locks the table exclusively for minutes/hours on large tables
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- GOOD: no table lock, can be done while production queries run
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders(customer_id);
```

### 4. Forgetting that functions break index usage

```
-- BAD: index on email is NOT used because of lower()
WHERE lower(email) = 'user@example.com'

-- FIX: create an expression index
CREATE INDEX ON users(lower(email));
```

### 5. Never vacuuming index bloat

Dead tuples accumulate in indexes. Without regular VACUUM, indexes grow bloated and slow. Enable `autovacuum` and monitor `pg_stat_user_tables.n_dead_tup`.

---

## Best Practices

1. **Index foreign keys** — PostgreSQL does not auto-index FK columns. Without them, every FK constraint check and JOIN on the referenced column triggers a Seq Scan.
2. **Use CONCURRENTLY** for all production index operations.
3. **Monitor pg\_stat\_user\_indexes** weekly — drop indexes with `idx_scan = 0` after a representative period (e.g., 30 days).

4. Set `random_page_cost = 1.1` on SSD storage so the planner correctly prefers index scans over sequential scans.
5. **Prefer composite indexes over multiple single-column indexes** when queries frequently filter on multiple columns together.
6. **Use partial indexes** to index only the "hot" subset of rows (e.g., unprocessed queue items, active records).
7. **Test with realistic data volumes** — an index that helps on a 10K-row dev database may be skipped by the planner on a 100M-row production database if statistics differ.

---

## Performance Considerations

### Effect of `random_page_cost` on index choice

```
-- Check current settings
SHOW random_page_cost;
SHOW seq_page_cost;

-- For SSDs (highly recommended)
ALTER SYSTEM SET random_page_cost = 1.1;
SELECT pg_reload_conf();
```

### `effective_cache_size` — tell the planner how much OS cache is available

```
-- Default: 4GB. Set to ~75% of total RAM for accurate cost estimates.
ALTER SYSTEM SET effective_cache_size = '24GB';
SELECT pg_reload_conf();
```

When `effective_cache_size` is higher the planner assumes more index pages are cached and prefers index scans more aggressively.

### `work_mem` and index usage

Higher `work_mem` makes hash joins and sort operations cheaper relative to index scans, potentially causing the planner to prefer them over index-based plans. Tune carefully.

---

## Interview Questions & Answers

### Q1. What is an index and what problem does it solve?

A: An index is a separate data structure that maps column values to heap row locations (ctids). It solves the sequential scan problem: without an index, finding rows matching a predicate requires reading every page of the table. An index reduces that to  $O(\log n)$  lookups for B-tree, or  $O(1)$  for hash indexes.

---

### Q2. What are the three main costs introduced by adding an index?



A: (1) Storage cost — the index occupies disk space and buffer cache. (2) Write cost — every INSERT, UPDATE, or DELETE on indexed columns must also update the index. (3) Planner overhead — more indexes means more plan alternatives to evaluate, though this is usually negligible.

---

### Q3. How does PostgreSQL decide whether to use an index or do a sequential scan?

A: Through its cost-based optimizer. It estimates the number of rows matching the predicate (selectivity) using statistics from `pg_statistic`, then calculates the cost of each plan (index scan vs seq scan) using cost parameters like `random_page_cost` and `seq_page_cost`. It chooses the plan with the lowest estimated total cost. For low selectivity (few matching rows) index scans win; for high selectivity (many matching rows) sequential scans win because sequential I/O is cheaper than many random I/Os.

---

### Q4. Why should you almost always use CREATE INDEX CONCURRENTLY in production?

A: A standard `CREATE INDEX` acquires a ShareLock on the table, which blocks all writes (INSERT, UPDATE, DELETE) for the duration of index construction. On a large table this can take minutes or hours. `CREATE INDEX CONCURRENTLY` builds the index in multiple passes while allowing concurrent reads and writes, with only very brief locks at the end. The tradeoff is slightly longer total build time and the possibility that it fails partway through (leaving an INVALID index that must be dropped and re-created).

---

### Q5. What happens when a WHERE clause uses a function on an indexed column?

A: The index is not used. For example, `WHERE lower(email) = 'foo@bar.com'` will not use an index on `email`. To fix this, create an **expression index**: `CREATE INDEX ON users(lower(email))`. The planner recognizes that the expression in the query matches the expression in the index and uses it.

---

### Q6. What is index selectivity and how does it relate to index usefulness?

A: Selectivity is the fraction of rows a predicate returns (0 = no rows, 1 = all rows). High selectivity (value close to 1, many rows) = index not useful because the planner would need to do many random reads anyway. Low selectivity (value close to 0, few rows) = index very useful because it precisely identifies a small subset of heap pages to read. The crossover point is typically around 1–5% depending on `random_page_cost`.

---

### Q7. How do you find unused indexes in PostgreSQL?

A: Query `pg_stat_user_indexes` and filter for `idx_scan = 0`. Also check the `idx_tup_read` and `idx_tup_fetch` columns. Be careful to collect statistics over a representative time period (at least 30 days including peak load) before dropping indexes.

---

### Q8. Why should foreign key columns be indexed in PostgreSQL?

A: PostgreSQL does not automatically create indexes on foreign key columns. When a referenced row is deleted or updated, PostgreSQL must check all child rows to enforce referential integrity — this check is a sequential scan if no FK index exists. Additionally, JOINS on FK columns will be sequential scans. This can cause severe performance degradation on large tables.

---

### Q9. What is effective\_cache\_size and how does it affect index usage?

A: `effective_cache_size` tells the planner how much total memory (`shared_buffers` + OS page cache) is available for caching data. A higher value makes the planner assume more index pages are already in cache, reducing the estimated cost of index scans and making the planner prefer them more aggressively. It should be set to approximately 75% of total system RAM.

---

#### Q10. What is the difference between a unique index and a primary key?

A: A PRIMARY KEY constraint automatically creates a unique index and also adds a NOT NULL constraint. A UNIQUE index allows NULL values (and multiple NULLs are considered distinct by default). Functionally both enforce uniqueness and both create a B-tree index that the planner can use for lookups and joins.

---

#### Q11. Can an index slow down a query?

A: Yes, in several situations: (1) If the index is on a low-cardinality column and the planner chooses it anyway (due to stale statistics), causing more random I/O than a sequential scan would. (2) Bloated indexes with many dead tuples cause more pages to be read. (3) With `random_page_cost` set too high on SSD systems the planner may avoid indexes unnecessarily. Always verify with EXPLAIN ANALYZE.

---

#### Q12. What is index bloat and how is it addressed?

A: Index bloat occurs when many dead tuples (from UPDATES and DELETES) accumulate in index pages. The index grows larger but contains less useful data, causing more I/O per query. It is addressed by: (1) Regular VACUUM (removes dead index entries), (2) REINDEX to completely rebuild the index, (3) REINDEX CONCURRENTLY (PostgreSQL 12+) for non-blocking rebuilds, (4) Tuning autovacuum settings to run more frequently on high-churn tables.

---

## Exercises with Solutions

### Exercise 1

You have a table `products` with 5 million rows. A query `SELECT * FROM products WHERE category_id = 10 AND price < 100` runs slowly. Design the optimal index.

**Solution:**

```
-- Composite index with equality column first (more selective leading column first),
-- range column second
CREATE INDEX CONCURRENTLY idx_products_cat_price
ON products(category_id, price)
WHERE price < 1000; -- optional partial index if most queries have an upper bound

-- Verify with EXPLAIN ANALYZE
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM products WHERE category_id = 10 AND price < 100;
```

### Exercise 2

Write a query to identify all indexes on the `orders` table that have never been used since the last statistics reset.

**Solution:**

```
SELECT
    i.indexname,
    i.indexdef,
    s.idx_scan,
    pg_size_pretty(pg_relation_size(s.indexrelid)) AS size
```

```
FROM pg_indexes i
JOIN pg_stat_user_indexes s
  ON i.indexname = s.indexname
  AND i.tablename = s.relname
WHERE i.tablename = 'orders'
  AND s.idx_scan = 0
ORDER BY pg_relation_size(s.indexrelid) DESC;
```

### Exercise 3

The query `SELECT * FROM users WHERE email = 'alice@example.com'` ignores the index on `email`. Diagnose and fix.

#### Solution:

```
-- Step 1: Check EXPLAIN
EXPLAIN SELECT * FROM users WHERE email = 'alice@example.com';

-- Likely cause 1: Function on column
-- If query was actually: WHERE lower(email) = 'alice@example.com'
CREATE INDEX ON users(lower(email));

-- Likely cause 2: Type mismatch (email is varchar, value is text)
-- Usually auto-cast but check column types

-- Likely cause 3: Statistics are stale
ANALYZE users;

-- Likely cause 4: random_page_cost too high (on SSD)
SET random_page_cost = 1.1;
EXPLAIN SELECT * FROM users WHERE email = 'alice@example.com';
```

---

## Production Scenarios

### Scenario 1: Startup Scaling Problem

A startup's `events` table has grown from 100K to 50M rows over 6 months. Queries that ran in 10ms now take 45 seconds. Investigation reveals no indexes were ever added.

#### Action plan:

```
-- 1. Identify most common query patterns from slow query log
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;

-- 2. Add indexes concurrently for top query patterns
CREATE INDEX CONCURRENTLY idx_events_user_id ON events(user_id);
CREATE INDEX CONCURRENTLY idx_events_created ON events(created_at);
CREATE INDEX CONCURRENTLY idx_events_type_created ON events(event_type, created_at);
```

```
-- 3. Update planner statistics
ANALYZE events;

-- 4. Verify improvement
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM events WHERE user_id = 123 ORDER BY
created_at DESC LIMIT 10;
```

## Scenario 2: Too Many Indexes Hurting Writes

An e-commerce platform notices that order INSERT rate has dropped from 10,000/sec to 3,000/sec after a "performance optimization" sprint that added 15 new indexes to `orders` .

### Action plan:

```
-- 1. Find unused indexes
SELECT indexname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes
WHERE relname = 'orders' AND idx_scan < 100 -- rarely used
ORDER BY idx_scan;

-- 2. Drop them concurrently
DROP INDEX CONCURRENTLY idx_orders_rarely_used;

-- 3. Monitor write throughput improvement
SELECT * FROM pg_stat_user_tables WHERE relname = 'orders';
```

---

## Cross-References

- [02 btree indexes.md](#) — B-tree structure in detail
- [08 partial indexes.md](#) — Partial index use cases
- [09 composite indexes.md](#) — Column ordering rules
- [12 index maintenance.md](#) — REINDEX, bloat, monitoring
- [../08\\_Query\\_Optimization/01\\_explain\\_basics.md](#) — Reading EXPLAIN output
- [../08\\_Query\\_Optimization/05\\_query\\_planner.md](#) — Planner statistics